# Practical 4

*Convolutional Neural Network (CNN) for Image Classification and Comparison with a Basic ANN*

---

## Aim

To build, train and evaluate a Convolutional Neural Network (CNN) using Conv2D and MaxPooling2D layers on the same dummy image dataset used in Practical 3, and to compare its performance against the basic Dense (fully connected) neural network.

## Conceptual / Theoretical Background

A Convolutional Neural Network (CNN) is a type of neural network designed specifically for grid-like data such as images. Unlike a Dense network, which flattens the image and treats every pixel as an independent input, a CNN keeps the image in its 2D (or 3D, with channels) form and learns small filters (kernels) that slide across the image, detecting local patterns such as edges, corners and textures.

A Conv2D layer learns a set of filters; each filter produces a 'feature map' highlighting where a particular pattern occurs in the image. Because the same filter is applied across the whole image (weight sharing), CNNs need far fewer parameters than an equivalent Dense layer and are naturally sensitive to spatial structure.

A MaxPooling2D layer downsamples each feature map by keeping only the maximum value in each small window (e.g. 2×2), which reduces the spatial size, makes the network more robust to small shifts in the pattern's position, and reduces computation for later layers.

Dropout is a regularisation technique that randomly disables a fraction of neurons during each training step, forcing the network to not rely too heavily on any single neuron and helping to reduce overfitting — CNNs, having more representational capacity, are more prone to overfitting small datasets than a simple Dense network.

Conv2D layers expect input with an explicit channel dimension (height, width, channels), so the grayscale images (originally 28×28) must be reshaped to (28, 28, 1) before being fed into the CNN.

## Real-World Applications

- Large-scale image classification systems (e.g. ImageNet-style classifiers) are built almost exclusively on CNN backbones.
- Facial recognition and biometric verification systems use CNNs to extract discriminative facial features.
- Medical image diagnosis (X-ray, MRI, histopathology) uses CNNs to detect and classify abnormalities.
- Self-driving car perception systems use CNNs for lane detection, traffic-sign recognition and object classification.
- CNNs also form the backbone (feature extractor) of modern object-detection systems such as YOLO and Faster R-CNN, which is the direction the next practical (Practical 5) moves toward.

## Important Terminology

| Term | Meaning |
|---|---|
| **Convolutional layer (Conv2D)** | A layer that learns filters which slide over the input to detect local spatial patterns. |
| **Filter / Kernel** | A small learnable weight matrix (e.g. 3×3) applied by a convolutional layer to detect a specific pattern. |
| **Feature map** | The output produced when a filter is convolved across the input; highlights where a pattern was found. |
| **Stride** | The step size by which a filter moves across the input (default 1 unless specified). |
| **Pooling / MaxPooling2D** | A downsampling operation that keeps the maximum value in each local window, reducing spatial size. |
| **Dropout** | A regularisation technique that randomly disables neurons during training to reduce overfitting. |
| **Overfitting** | When a model learns the training data (including its noise) too closely and performs worse on unseen data. |
| **Channel dimension** | The extra axis (e.g. 1 for grayscale, 3 for RGB) that Conv2D layers require in the input shape. |

## Step-by-Step Working

1. Import the additional CNN-specific Keras layers: Conv2D, MaxPooling2D, and Dropout.
2. Reshape the already-normalized and already-split X_train / X_test arrays from Practical 3 (shape (28, 28)) into (28, 28, 1) to add the required channel dimension; y_train / y_test are reused unchanged.
3. Build a Sequential CNN: Conv2D(16 filters) → MaxPooling2D → Conv2D(32 filters) → MaxPooling2D → Flatten → Dense(64, ReLU) → Dropout(0.3) → Dense(3, Softmax).
4. Compile the CNN with the same optimizer, loss and metric as the basic network (Adam, sparse categorical cross-entropy, accuracy) so the two models are directly comparable.
5. Train the CNN for 10 epochs with batch size 32, using the same validation split as before.
6. Plot the CNN's training vs. validation accuracy across epochs.
7. Evaluate the CNN on the test set to obtain its test loss and accuracy.
8. Print a side-by-side comparison of the basic Dense network's and the CNN's test loss/accuracy.
9. Predict the class of a few test images using the CNN and visually compare predicted vs. true labels.
10. Save the trained CNN model to a separate file from the Dense model.

## Python Implementation

1. Reshape data for Conv2D input:

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dropout

# X_train, X_test, y_train, y_test are reused from Practical 3
X_train_cnn = X_train.reshape(-1, img_size, img_size, 1)
X_test_cnn = X_test.reshape(-1, img_size, img_size, 1)

print("CNN training data shape:", X_train_cnn.shape)
print("CNN testing data shape :", X_test_cnn.shape)
```

2. Build, compile and train the CNN:

```
cnn_model = Sequential([
    Conv2D(16, (3, 3), activation='relu', input_shape=(img_size, img_size, 1)),
    MaxPooling2D((2, 2)),
    Conv2D(32, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(3, activation='softmax')
])
cnn_model.summary()

cnn_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

cnn_history = cnn_model.fit(
    X_train_cnn, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

3. Evaluate and compare against the Dense network:

```
cnn_test_loss, cnn_test_accuracy = cnn_model.evaluate(X_test_cnn, y_test, verbose=0)
print(f"CNN Test Loss     : {cnn_test_loss:.4f}")
print(f"CNN Test Accuracy : {cnn_test_accuracy:.4f}")

print("Model Comparison")
print("-" * 40)
print(f"Basic NN  -> Loss: {test_loss:.4f}, Accuracy: {test_accuracy:.4f}")
print(f"CNN       -> Loss: {cnn_test_loss:.4f}, Accuracy: {cnn_test_accuracy:.4f}")
```

4. Predict and save:

```
cnn_predictions = cnn_model.predict(X_test_cnn[:5])
class_names = ['Square', 'Diagonal', 'Horizontal']

for i in range(5):
    predicted_label = np.argmax(cnn_predictions[i])
    true_label = y_test[i]
    print(f"Predicted: {class_names[predicted_label]}  |  True: {class_names[true_label]}")

cnn_model.save("cnn_dummy_image_model.keras")
print("CNN model saved successfully.")
```

## Explanation of the Code

- X_train.reshape(-1, img_size, img_size, 1) adds a trailing channel dimension of size 1 (grayscale) without changing the pixel data or the number of samples, satisfying Conv2D's expected input shape.

- The first Conv2D(16, (3,3)) layer learns 16 different 3×3 filters over the raw image, and MaxPooling2D((2,2)) halves the spatial resolution while keeping the strongest activations; the second Conv2D(32, (3,3)) + MaxPooling2D pair repeats this to learn higher-level combinations of the first layer's features.
- Flatten() converts the final stack of small feature maps into a 1D vector so it can be passed into the Dense(64, ReLU) layer, matching the Dense-network structure from Practical 3 at this point.
- Dropout(0.3) randomly zeroes out 30% of the Dense layer's activations during each training step, which is included because CNNs have more capacity than the basic network and can otherwise overfit this small dataset more easily.
- The CNN is compiled and trained with exactly the same optimizer, loss function, batch size and number of epochs as the Dense network from Practical 3, so any difference in the final accuracy reflects the architecture change rather than the training setup.
- The side-by-side print statement directly compares cnn_test_accuracy against the test_accuracy computed for the Dense model in Practical 3.
- cnn_model.save() writes the CNN to a separate .keras file (cnn_dummy_image_model.keras) so it does not overwrite the Dense model saved in Practical 3.

## Expected Output

A typical run (results vary slightly with random initialisation) produces:

```
CNN training data shape: (480, 28, 28, 1)
CNN testing data shape : (120, 28, 28, 1)

Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 26, 26, 16)        160
 max_pooling2d               (None, 13, 13, 16)        0
 conv2d_1 (Conv2D)           (None, 11, 11, 32)        4640
 max_pooling2d_1             (None, 5, 5, 32)          0
 flatten_1 (Flatten)         (None, 800)               0
 dense_3 (Dense)             (None, 64)                51264
 dropout (Dropout)           (None, 64)                0
 dense_4 (Dense)             (None, 3)                 195
=================================================================
Total params: 56,259

Epoch 1/10 ... accuracy: 0.62 - val_accuracy: 0.85
...
Epoch 10/10 ... accuracy: 1.00 - val_accuracy: 1.00

CNN Test Loss     : 0.0187
CNN Test Accuracy : 1.0000

Model Comparison
----------------------------------------
Basic NN  -> Loss: 0.0542, Accuracy: 0.9833
CNN       -> Loss: 0.0187, Accuracy: 1.0000

Predicted: Square      |  True: Square
Predicted: Diagonal    |  True: Diagonal
Predicted: Horizontal  |  True: Horizontal
Predicted: Square      |  True: Square
Predicted: Diagonal    |  True: Diagonal
```

```
CNN model saved successfully.
```

## Conclusion

The CNN matched or slightly outperformed the basic Dense network on this simple synthetic dataset while learning far fewer parameters per layer through weight sharing, and it did so by exploiting the 2D spatial structure of the images rather than treating every pixel independently. On more complex, real-world image datasets this advantage becomes much larger, which is why CNNs (and their successors) are the standard architecture for image classification and form the backbone of the object-detection pipeline explored in Practical 5.